Type–length–value

Within communication protocols, **TLV** (**type-length-value** or **tag-length-value**) is an encoding scheme used for informational elements. A TLV-encoded data stream contains code related to the record type, the record value's length, and finally the value itself.

Details

The type and length are fixed in size (typically 1–4 bytes) or can be otherwise parsed without knowledge of the size (see: LEB128, variable-length quantity), and the value field is of variable size. These fields are used as follows:

Type
: A binary code, often simply alphanumeric, which indicates the kind of field that this part of the message represents;

Length
: The size of the value field (typically in bytes);

Value
: Variable-sized series of bytes which contains data for this part of the message.

Some advantages of using a TLV representation data system solution are:

- TLV sequences are easily searched using generalized parsing functions;
- New message elements which are received at an older node can be safely skipped and the rest of the message can be parsed. This is similar to the way that unknown XML tags can be safely skipped;
- TLV elements can be placed in any order inside the message body;
- TLV elements are typically used in a binary format and binary protocols which makes parsing faster and the data smaller than in comparable text based protocols.

Illustrative example

Imagine a message to make a telephone call. A first version of the system defines the following structure:

```
struct message {
  uint16_t tag;
  uint16_t length;
  char value[length];
}
/* Tags */
#define T_COMMAND                0x00
#define T_PHONE_NUMBER_TO_CALL   0x10
/* Command values */
#define C_MAKE_CALL              0x20
```

When it makes a call, it sends the following data:

```
00 00          T_COMMAND
00 04          length = 4
00 00 00 20    C_MAKE_CALL
00 10          T_PHONE_NUMBER_TO_CALL
00 08          length = 8
37 32 32 2D    ASCII for "722-"
34 32 34 36    ASCII for "4246"
```

A receiving system would then understand that the message tells it to call "722-4246".

Later (in version 2) a new field containing the calling number could be added:

```
#define T_CALLER_NUMBER           0x11
```

It would send a message like:

```
00 00          T_COMMAND
00 04          length = 4
00 00 00 20    C_MAKE_CALL
00 11          T_CALLER_NUMBER
00 0c          length = 12
36 31 33 2D    ASCII for "613-"
37 31 35 2D    ASCII for "715-"
39 37 31 39    ASCII for "9719"
00 10          T_PHONE_NUMBER_TO_CALL
00 08          length = 8
37 32 32 2D    ASCII for "722-"
34 32 34 36    ASCII for "4246"
```

A version 1 system which received a message from a version 2 system would first read the `T_COMMAND` element and then read an element of type `T_CALLER_NUMBER`. The version 1 system does not understand `T_CALLER_NUMBER`, so the length field is read (i.e. 12) and the system skips forward 12 bytes to read `T_PHONE_NUMBER_TO_CALL`, which it understands, and message parsing carries on.

Real-world examples

Transport protocols

- TLS (and its predecessor SSL) use TLV-encoded messages.
- SSH
- COPS
- IS-IS
- RADIUS
- Link Layer Discovery Protocol allows for the sending of organizational-specific information as a TLV element within LLDP packets
- Media Redundancy Protocol allows organizational-specific information
- Dynamic Host Configuration Protocol (DHCP) uses TLV encoded options
- RR protocol used in GSM cell phones (defined in 3GPP 04.18). In this protocol each message is defined as a sequence of information elements.

Data storage formats

- IFF
- Matroska uses TLV for markup tags
- QTFF (the basis for MPEG-4 containers)

Other

- ubus used for IPC in OpenWrt[1]

Other ways of representing data

Core TCP/IP protocols (particularly IP, TCP, and UDP) use predefined, static fields.

Some application layer protocols, including HTTP/1.1 (and its non-standardized predecessors), FTP, SMTP, POP3, and SIP, use text-based "Field: Value" pairs formatted according to RFC 2822 (https://www.rfc-editor.org/rfc/rfc2822). (HTTP represents length of payload with a Content-Length header and separates headers from the payload with an empty line and headers from each other with a new line.)

ASN.1 specifies several TLV-based encoding rules (BER, DER), as well as non-TLV based ones (PER, XER, JSON Encoding Rules). The TLV-based rules can be parsed without knowing the possible members of the message, while the non-TLV/static PER cannot. XER uses XML, which also allows for parsing without knowing the possible members of the message; the same applies to the JSON encoding rules.

CSN.1 describes encoding rules using non-TLV semantics.

More recently, XML has been used to implement messaging between different nodes in a network. These messages are typically prefixed with line-based text commands, such as with BEEP.

See also

- KLV, specific type of type-length-value encoding

References

1. "OpenWrt documentation on ubus" (https://openwrt.org/docs/techref/ubus). *openwrt.org*. April 15, 2022. Retrieved 2022-04-15.